

URL: http://www.sommarskog.se/share_data.html

How to Share Data between Stored Procedures

An SQL text by [Erland Sommarskog](#), SQL Server MVP. [Most recent update](#) 2013-11-02.

Introduction

This article tackles two related questions:

- *How can I use the result set from one stored procedure in another, also expressed as How can I use the result set from a stored procedure in a SELECT statement?*
- *How can I pass a table data in a parameter from one stored procedure to another?*

In this text I will discuss a number of possible solutions and point out their advantages and drawbacks. Some methods apply only when you want to access the output from a stored procedure, whereas other methods are good for the input scenario, or both input and output. In the case you want to access a result set, most methods require you to rewrite the stored procedure you are calling (the *callee*) in one way or another, but some solutions do not.

Here is a summary of the methods that I will cover. *Required version* refers to the minimum version of SQL Server where the solution is available. When the column is empty, this means all versions from SQL 2000 and up.

Method	In/Out	Rewrite callee?	Required version	Comment
OUTPUT Parameters	Output	Yes		Not generally applicable, but sometimes overlooked.
Table-valued Functions	Output	Yes		Often the best choice for output-only, but there are several restrictions.
Inline Functions				Use this to reuse a single SELECT.
Multi-statement Functions				When you need to encapsulate more complex logic.
Using a Table	In/Out	Yes		The most general solution. My favoured choice for input/output scenarios.
Sharing a Temp Table				Mainly for a single pair of caller/callee.

<u>Process-keyed Table</u>				Best choice for many callers to the same callee.
<u>Global Temp Tables</u>				A variation of process-keyed.
<u>Table-valued Parameters</u>	Input	Yes	SQL 2008	Mainly useful when passing data from a client.
<u>INSERT-EXEC</u>	Output	No		Deceivingly appealing, but should be used sparingly.
<u>Using the CLR</u>	Output	No	SQL 2005	Complex, but useful as a last resort when INSERT-EXEC does not work.
<u>OPENQUERY</u>	Output	No		Tricky with many pitfalls. Discouraged.
<u>Using XML</u>	In/Out	Yes	SQL 2005	A bit of a kludge, but not without advantages.
<u>Using Cursor Variables</u>	Output	Yes		Not recommendable.

At the end of the article, I briefly discuss the particular situation when your stored procedures are on [different servers](#), which is a quite challenging situation.

A related question is how to pass table data from a client, but this is a topic which is outside the scope for this text. Of the methods that I discuss in this article, only table-valued parameters and XML are useful for this case. For a more general discussion on passing structured data from a client to SQL Server, see my article [Arrays and Lists in SQL Server](#).

Examples in the article featuring tables such as *authors*, *titles*, *sales* etc run in the old sample database *pubs*. You can download the script for pubs from [Microsoft's web site](#). (Some examples use purely fictive tables, and do not run in pubs.)

OUTPUT Parameters

This method can only be used when the result set is one single row. Nevertheless, this is a method that is sometimes overlooked. Say you have this simple stored procedure:

```
CREATE PROCEDURE insert_customer @name nvarchar(50), @address nvarchar(50), @city nvarchar(50)
AS DECLARE @cust_id int BEGIN TRANSACTION SELECT @cust_id = coalesce(MAX(cust_id), 0) + 1 FROM
customers (UPDLOCK) INSERT customers (cust_id, name, address, city) VALUES (@cust_id, @name,
@address, @city) COMMIT TRANSACTION SELECT @cust_id
```

That is, the procedure inserts a row into a table, and returns the id for the row.

Rewrite this procedure as:

```
CREATE PROCEDURE insert_customer @name nvarchar(50), @address nvarchar(50), @city nvarchar(50),  
@cust_id int OUTPUT AS BEGIN TRANSACTION SELECT @cust_id = coalesce(MAX(cust_id), 0) + 1 FROM  
customers (UPDLOCK) INSERT customers (cust_id, name, address, city) VALUES (@cust_id, @name,  
@address, @city) COMMIT TRANSACTION
```

You can now easily call **insert_customer** from another stored procedure. Just recall that in T-SQL you need to specify the OUTPUT keyword also in the call:

```
EXEC insert_customer @name, @address, @city, @cust_id OUTPUT
```

Note: this example has a single output parameter, but a stored procedure can have many output parameters.

Table-valued Functions

When all you want to do is to reuse the result set from a stored procedure, the first thing to investigate is whether it is possible to rewrite the stored procedure as a table-valued function. This is far from always possible, because SQLServer is very restrictive with what you can put into a function. But when it is possible, this is often the best choice.

There are two types of table functions in SQL Server: inline and multi-statement functions.

Inline Functions

Here is a example of an inline function adapted from Books Online for SQL 2000:

```
CREATE FUNCTION SalesByStore (@storeid varchar(30)) RETURNS TABLE AS RETURN (SELECT t.title, s.qty  
FROM sales s JOIN titles t ON t.title_id = s.title_id WHERE s.stor_id = @storeid)
```

To use it, you simply say:

```
SELECT * FROM SalesByStore('6380')
```

You can filter the data with WHERE or use it in a bigger query that includes other tables. That is, you use the function just like was a table or a view. You could say that an inline function is a parameterised view, because the query optimizer expands the function as if it was a macro, and generates the plan as if you had provided the expanded query. Thus, there is no performance cost for packaging a SELECT statement into an inline function. For this reason, when you want to reuse a stored procedure that consists of a single SELECT statement, rewriting it into an inline UDF is without doubt the best choice. (Or instead of rewriting it, move the SELECT into a UDF, and rewrite the existing procedure as a wrapper on the function, so that the client is unaffected.)

There are a couple of system functions you cannot use in a UDF, because SQL Server thinks it matters that they are *side-effecting*. The most commonly used ones are **newid()**, and **rand()**. On SQL 2000 this restriction goes further and disallows all system functions that are *nondeterministic*, that is, functions that do not return the same value for the same input parameters on each call. A typical example is **getdate()**.

Multi-statement Functions

A multi-statement function has a body that can have as many statements as you like. You need to declare a return table, and you insert the data to return into that table. Here is the function above as a multi-statement function:

```
CREATE FUNCTION SalesByStore (@storeid varchar(30)) RETURNS @t TABLE (title varchar(80) NOT NULL
PRIMARY KEY, qty smallint NOT NULL) AS BEGIN INSERT @t (title, qty) SELECT t.title, s.qty FROM sales s
JOIN titles t ON t.title_id = s.title_id WHERE s.stor_id = @storeid RETURN END
```

You use multi-statement functions in the same way as you use inline functions, but in difference to inline functions, they are not expanded in place, but instead it's like you would call a stored procedure in the middle of the query and return the data in a table variable. This permits you to move the code of a more complex stored procedure into a function.

As you can see in the example, you can define a primary key for your return table. I like to point out that this definitely best practice for two reasons:

1. It states your assumptions of the data. If your assumptions are incorrect, you will be told up front. (Instead of spending time to understand why your application presents incorrect data.)
2. This is information that is valuable to the optimizer when you use the function in a larger query.

It goes without saying, that this is only meaningful if you define a primary key on the columns you produce in the body of the UDF. Adding an IDENTITY column to the return table only to get a primary key is pointless.

Compared to inline functions, multi-statement functions incur some overhead due to the return table. More important, though, is that if you use the function in a query where you join with other tables, the optimizer will have no idea of what the function returns, and will make standard assumptions. This is far from always an issue, but the more rows the function returns, the higher the risk that the optimizer will make incorrect estimates and produce an inefficient query plan. One way to avoid this is to insert the results from the function into a temp table. Since a temp table has statistics this helps the optimizer to make a better plan.

It follows from this, that there is not much reason to consider which sort of function to use. If you can express your problem in a single query, use an inline function. Only use a multi-statement function when an inline function is not possible.

User-defined functions are quite restricted in what they can do, because a UDF is not permitted to change the database state. The most important restrictions are:

- You can only perform INSERT, UPDATE or DELETE statements on table variables local to the function.
- You cannot call stored procedures (with the exception of extended stored procedures).
- You cannot invoke dynamic SQL.
- You cannot create tables, neither permanent tables nor temp tables. You can use table variables.
- You cannot use RAISERROR, TRY-CATCH or BEGIN/COMMIT/ROLLBACK TRANSACTION.
- You cannot use "side-effecting" system functions, such as **newid()** and **rand()**.
- On SQL 2000, you cannot use non-deterministic system functions.

Please see the Remarks section in the topic for CREATE FUNCTION in Books Online for a complete list of restrictions.

Using a Table

What could be better for passing data in a database than a table? When using a table there are no restrictions, but you have a solution that works in all situations. We will look at two ways to do this, as well as a third way which is a variation of the second. It should be admitted, though, that this is a little more heavy-handed than some of the other solutions in this article. Using tables can also lead to performance issues due to recompilation.

Sharing a Temp Table

Introduction

The method itself is as simple as this:

```
CREATE PROCEDURE called_procedure @par1 int, @par2 bit, ... AS ... INSERT/UPDATE/DELETE #tmp go
CREATE PROCEDURE caller AS DECLARE ... CREATE TABLE #mytemp (col1 int NOT NULL, col2 char(5)
NULL, ...) ... EXEC called_procedure @par1, @par2 ... SELECT * FROM #mytemp go
```

In this example, **caller** creates the temp table, and **called_procedure** fills it in, that is, the table is output-only. A different scenario is that **caller** fills the table with input data whereupon **called_procedure** performs some general computation, and the caller uses the result from that computation for some purpose. That is, the table is used for both input and output. Yet a scenario is that the caller prepares the temp table with data, and the callee first performs checks to verify that a number of business rules are not violated, and then goes on to update one or more tables. This would be an input-only scenario.

Changing Existing Code

Say that you have this procedure:

```
CREATE PROCEDURE SalesByStore @storeid varchar(30) AS SELECT t.title, s.qty FROM sales s JOIN titles t
ON t.title_id = s.title_id WHERE s.stor_id = @storeid
```

You want to reuse this result set in a second procedure that returns only titles that have sold above a certain quantity. How would you achieve this by sharing a temp table without affect existing clients? The solution is to move the meat of the procedure into a sub-procedure, and make the original procedure a wrapper on the original like this:

```
CREATE PROCEDURE SalesByStore_core @storeid varchar(30) AS INSERT #SalesByStore (title, qty)
SELECT t.title, s.qty FROM sales s JOIN titles t ON t.title_id = s.title_id WHERE s.stor_id = @storeid go
CREATE PROCEDURE SalesByStore @storeid varchar(30) AS CREATE TABLE #SalesByStore(title
varchar(80) NOT NULL PRIMARY KEY, qty smallint NOT NULL) EXEC SalesByStore_core @storeid SELECT *
FROM #SalesByStore go CREATE PROCEDURE BigSalesByStore @storeid varchar(30), @qty smallint AS
CREATE TABLE #SalesByStore(title varchar(80) NOT NULL PRIMARY KEY, qty smallint NOT NULL) EXEC
SalesByStore_core @storeid SELECT * FROM #SalesByStore WHERE qty >= @qty go EXEC SalesByStore
'7131' EXEC BigSalesByStore '7131', 25 go DROP PROCEDURE SalesByStore, BigSalesByStore,
SalesByStore_core
```

Note: This script is a complete *repro* script that creates some objects, tests them, and then drops them, to permit simple resting of variations. We will look at more versions of these procedures later in this text.

Just like in the example with the multi-statement function, I have defined a primary key for the temp table, and exactly for the same reasons. Speaking of best practices, some readers may wonder about the use of `SELECT *` here. I think using `SELECT *` from a temp table created in the same procedure is OK, particularly if the purpose is to return all columns in the temp table. (In difference to using `SELECT *` from a table created elsewhere, and which may be altered without your knowledge.)

While this solution is straightforward, you may feel uneasy by the fact that the `CREATE TABLE` statement for the temp table appears in two places, and there is a third procedure that depends on the definition. Here is a solution which is a little more convoluted that to some extent alleviates the situation:

```
CREATE PROCEDURE SalesByStore_core @storeid varchar(30), @wantresultset bit = 0 AS IF
object_id('tempdb..#SalesByStore') IS NULL BEGIN CREATE TABLE #SalesByStore(title varchar(80) NOT
NULL PRIMARY KEY, qty smallint NOT NULL) END INSERT #SalesByStore (title, qty) SELECT t.title, s.qty
FROM sales s JOIN titles t ON t.title_id = s.title_id WHERE s.stor_id = @storeid IF @wantresultset = 1
SELECT * FROM #SalesByStore go CREATE PROCEDURE SalesByStore @storeid varchar(30) AS EXEC
SalesByStore_core @storeid, 1 go
```

I've moved the `CREATE TABLE` statement for the wrapper into the core procedure, which only creates the temp table only if it does not already exist. The wrapper now consists of a single `EXEC` statement and passes the parameter `@wantresultset` as 1 to instruct the core procedure to produce the result set. Since this parameter has a default of 0, `BigSalesByStore` can be left unaffected.

A Note on the Virtues of Code Reuse

Before we move on, I like to point out that the given example as such is not very good practice. Not because the concept of sharing temp tables as such is bad, but as with all solutions, you need to use them in the right place. As you realise, defining a temp table and creating one extra stored procedure is too heavy artillery for this simple problem. But an example where sharing temp tables would be a good solution would have to consist of many more lines of code, which would have obscured the forest with a number of trees. Thus, I've chosen a very simple example to highlight the technique as such.

Keep in mind that compared to languages such as C# and Java, Transact-SQL is poorly equipped for code reuse, why solutions in T-SQL to reuse code are clumsier. For this reason, the bar for reuse is somewhat higher in T-SQL. It's still a virtue, but not as big virtue as in modern object-oriented languages. In this simple problem, the best would of course be to add `@qty` as a parameter to `SalesByStore`. And if that would not be feasible for some reason, it would still be better to create `BigSalesByStore` by means of copy-paste than sharing a temp table.

Beside the poverty of T-SQL as a language, there is a second reason why code reuse is something that you should be a little careful with. Say that there is a requirement to show the sales for all stores in a certain state. In a pure C# or Java environment, it would be normal to write a loop that calls `SalesByStore` for every store. But in a database with several hundred gigabytes of data, the performance penalty for such a solution can be severe.

A Maintenance Problem

If the callee is called from many places, and you want to change which columns it reads/writes, you need to revisit all calling stored procedures to edit the temp-table definition. For this reason, sharing temp

tables is mainly useful when you have a single pair of caller and callee. Then again, if the temp is narrow, maybe only a single column of customer IDs to process, the table is likely to be very stable.

There are some alternatives to overcome the maintenance problem. One is to use a [process-keyed table](#), which we will look into in the next section. I have also received some interesting ideas from readers of this article.

One solution comes from Richard St-Aubin. The callers create the temp table with a single dummy column, and then call a stored procedure that uses ALTER TABLE to add the real columns. It would look something like this:

```
CREATE PROCEDURE called_procedure @par1 int, @par2 bit, ... AS ... INSERT/UPDATE/DELETE #mytemp
go CREATE PROCEDURE define_temp_table AS ALTER TABLE #mytemp ADD col1 int NOT NULL, col2
char(5) NULL, ... go CREATE PROCEDURE caller AS DECLARE ... CREATE TABLE #mytemp (dummycol bit)
EXEC define_temp_table ... EXEC called_procedure @par1, @par2 ... SELECT * FROM #mytemp go
```

You must create the temp table in **caller**, since if you were to put the CREATE TABLE statement in **define_temp_table**, the table would be dropped when that procedure exits. This method can definitely be worth exploring, but I like to add a word of caution. Adding columns to a table at run-time can lead to unexpected errors if the procedure is recompiled. If you call the procedure that adds the columns directly after the CREATE TABLE statement, you should be fairly safe. But this depends on the fine print in SQL Server, so it could break with a new release. A second issue is that this method prevents SQL Server from caching the temp-table definition. This could have a significant impact if the procedures are called with a high frequency.

Another solution, which requires SQL 2008, comes from Wayne Bloss. He creates a table type that holds the definition of the temp table. You can only use table types for declaring table variable and table parameters. But Wayne has a cure for this:

```
DECLARE @dummy my_table_type SELECT * INTO #mytemp FROM @dummy
```

From this point you work with #mytemp; the sole purpose of @dummy is to be able to create #mytemp from a known and shared definition. (If you are unacquainted with table types, we will take a closer look on them in the section on [table-valued parameters](#).) A limitation with this method is that you can only centralise column definitions this way, but not constraints as they are not copied with SELECT INTO. You may think that constraints are odd things you rarely put in a temp table, but I have found that it is often fruitful to add constraints to my temp tables as assertions for my assumptions about the data. This does not the least apply for temp tables that are shared between stored procedures. Also, defining primary keys for your temp tables can avoid performance issues when you start to join them.

Let me end this section by pointing out that sharing temp tables opens for an interesting possibility for flexibility. The callee only cares about the columns it reads or writes. This permits a caller to add extra columns for its own usage when it creates the temp table. Thus, two callers to the same callee could have different definitions of the temp table, as long as the columns accessed by the callee are defined consistently.

Note: A more advanced way to tackle the maintenance problem is to use a pre-processor and put the definition of the temp table in an include-file. If you have a C compiler around, you can use the C

pre-processor. My [AbaPerls](#) includes a pre-processor, [Preppis](#), which we use in the system I spend most of my time with.

The Impact of Recompilation

One distinct drawback with this method is that it causes a lot of recompilation in the callee. Each time the caller is invoked, a new instance of the temp table is created, and for this reason SQL Server must recompile all statements in the callee that refer to the temp table. (Recall what I said about flexibility in the previous paragraph. The definition could really be different.) If the execution time for the callee is expected to be subsecond and there are several complex statements in the procedure, the recompilation may add an overhead of more than 100 %. On the other hand, if the typical execution time of the callee is one minute, the cost of recompilation is likely to be negligible.

One way to reduce the amount of recompilation is to copy any input data in the shared table to a local table first thing, and then write back the result at the end. This restricts the recompilation to these two statements, but it goes without saying that this adds an overhead in itself, and it's mainly an option when you expect the table to hold a small amount of data.

I should hasten to add that recompilation can happen for several reasons. It is very common for temp tables to cause recompilation because of changed statistics, a topic I will return to when I discuss process-keyed tables.

Note: if you are still on SQL 2000, you should be aware of that in this version of SQL Server, recompilation is always on procedure level and therefore more expensive. Statement-level recompilation was introduced in SQL 2005.

A Note on SQL Server Data Tools

Simultaneously with SQL Server 2012, Microsoft released SQL Server Data Tools, SSDT. This is a very versatile environments that gives you many benefits. One benefit is that if you write a stored procedure like:

```
CREATE PROCEDURE example_sp AS CREATE TABLE #temp(a int NOT NULL) SELECT a FROM #temmp
```

SSDT will tell you up front of the misspelling about the temp table name, before you try to run the batch to create the procedure. This is certainly a very helpful feature, because a typo can be trapped early. However, SSDT has no notion about sharing temp tables, so SSDT will also give you a warning for a procedure like SalesByStore_core, or more precisely three: one per column. They are only warnings, so you can proceed, but it takes a handful of such procedures to clutter up the Error List window so there is a risk that you miss other and more important issues.

There is a way to suppress the warning: right-click the file in Solution Explorer and select Properties. There is a property *Suppress T-Sql Warning* and here you can enter the code for the error. But this means that you lose the checking of all table names in the procedure; there is no means to only suppress the warning only for the shared temp table.

All and all, if you are using SSDT, you will find an extra resistance barrier against sharing temp tables.

Process-Keyed Tables

This method evades the maintenance problem by using a permanent table instead. There is still a recompilation problem, though, but of a different nature.

Outline

A process-keyed table is simply a permanent table that serves as a temp table. To permit processes to use the table simultaneously, the table has an extra column to identify the process. The simplest way to do this is the global variable **@@spid** (**@@spid** is the process id in SQL Server). In fact, this is so common, that these tables are often referred to as *spid-keyed tables*. Here is an outline; I will give you a more complete example later.

```
CREATE TABLE process_keyed (spid int NOT NULL, col1 int NOT NULL, col2 char(5) NULL, ...) go
CREATE CLUSTERED INDEX processkey_ix ON process_keyed (spid) -- Add other columns as needed.
go ... DELETE process_keyed WHERE spid = @@spid
INSERT process_keyed (spi, col1, col2, ....) VALUES (@@spid, @val1, @val2, ...) ...
SELECT col1, col2, ... FROM process_keyed WHERE spid = @@spid ...
DELETE process_keyed WHERE spid = @@spid
```

A few things to note here:

3. The table should have a clustered index on the process key (**spid** in this example), as all queries against the table will include the condition **WHERE spid = @@spid**.
4. You should delete any existing data for **@@spid** before you insert any data into the table, as a safety precaution.
5. When you are finished using the data you should delete it, so that it does not occupy any extra space.

Choosing the Process-key

While it's common to use **@@spid** as the process key there are two problems with this:

6. If sloppy programmers neglect to clear the **spid** before and after use, old data may be passed to the callee, causing incorrect results that are difficult to understand how they arose.
7. If a client needs to pass a process-key around, there is no guarantee that it will have the same **spid** every time, since modern clients typically connect and disconnect for each call they make.

One alternative for the process-key is to use a GUID (data type **uniqueidentifier**). If you create the process key in SQL Server, you can use the function **newid()**. (You can rely on **newid()** to return a unique value, which is why it addresses the first point.) You may have heard that you should not have **guids** in your clustered index, but that applies when the **guid** is the primary key alone, since this can cause fragmentation and a lot of page splits. In a process-keyed table, you will typically have many rows for the same **guid**, so it is a different situation.

SQL 2012 offers a new alternative: get the process-key from a sequence, which is a new type of object in SQL 2012. A sequence is akin to an **IDENTITY** column, but it is an object of its own.

A Longer Example

Let's say that there are several places in the application where you need to compute the total number of sold books for one or more stores. You put this computation in a procedure **ComputeTotalStoreQty**, which operates on the table **stores_aid**. In this example, the procedure is nothing more than a simple **UPDATE** statement that computes the total number of books sold per store. A real-life problem could have a complex computation that runs over several hundred lines of code. There is also an example procedure **TotalStoreQty** which returns the total sales for a certain state. It fills **stores_aid** with all stores

in that state, calls ComputeTotalStoreQty and then returns the result to the client. Note that TotalStoreQty is still careful to clear its entry in stores_aid both before and after the call.

```
CREATE TABLE stores_aid (process_key uniqueidentifier NOT NULL, storeid char(4) NOT NULL, totalqty  
smallint NULL, CONSTRAINT pk_stores_aid PRIMARY KEY (process_key, storeid) ) go  
CREATE PROCEDURE ComputeTotalStoreQty @process_key uniqueidentifier AS  
UPDATE stores_aid SET totalqty = s.totalqty  
FROM stores_aid sa JOIN (SELECT stor_id, SUM(qty) AS totalqty FROM sales GROUP BY stor_id) AS s ON  
s.stor_id = sa.storeid WHERE sa.process_key = @process_key go  
CREATE PROCEDURE TotalStoreQty @state char(2) AS  
DECLARE @process_key uniqueidentifier  
SELECT @process_key = newid() DELETE stores_aid WHERE process_key = @process_key  
INSERT stores_aid(process_key, storeid) SELECT @process_key, stor_id FROM stores WHERE state = @state  
EXEC ComputeTotalStoreQty @process_key  
SELECT storeid, totalqty FROM stores_aid WHERE process_key = @process_key  
DELETE stores_aid WHERE process_key = @process_key go  
EXEC TotalStoreQty 'CA' go  
DROP PROCEDURE TotalStoreQty, ComputeTotalStoreQty  
DROP TABLE stores_aid
```

Please note that I have defined a proper key for stores_aid adhering to best practices.

Name Convention and Clean-up

You may wonder what that **_aid** in the table name comes from. In the environment where I do my daily chores, we have quite a few process-keyed tables, and we have adapted the convention that all these tables end in **-aid**. This way, when you read some code, you know directly that this is not a "real" table with persistent data. (Nevertheless some of our aid tables are very important in our system as they are used by core functions.)

There is a second point with this name convention. It cannot be denied that a drawback with process-keyed tables is that sloppy programmers could forget to delete data when they are done. Not only this wastes space, it can also result in incorrect row-count estimates leading to poor query plans. For this reason it is a good idea to clean up these tables on a regular basis. For instance, in our night-job we have a procedure that runs the query below and then executes the generated statements:

```
SELECT 'TRUNCATE TABLE ' + quotename(name) FROM sysobjects WHERE type = 'U' AND name LIKE  
'%aid'
```

Issues with Recompilation

As we saw, when sharing temp tables, this causes recompilations in the callee, because the temp table is a new table every time. While this issue does not exist with process-keyed tables, you can still get a fair share of recompilation because of auto-statistics, a feature which is enabled in SQL Server by default. For a permanent table, auto-statistics kicks in when the first 500 rows have been added, and then every time 20 % of the rows have changed. (For full details on recompilation, see this [white paper](#) by Eric Hanson and Yavor Angelov.) Since a process-keyed table is typically empty when it is not in use, auto-statistics sets in often. Sometimes this can be a good thing, as the statistics may help the optimizer to find a better plan. But as I noted previously, recompilation may also cause an unacceptable performance overhead.

As when sharing temp tables, one way to circumvent the recompilation is to copy data to a local table on input and copy back on output. But for process-keyed tables there are two more options:

8. Disable auto-statistics for the table entirely with **sp_autostats**.

9. Use the query hint `OPTION (KEEPFIXED PLAN)` for queries which are costly to recompile, and where the changed statistics are unlikely to affect the outcome of the compilation.

The Cost of Logging

Compared to sharing temp tables, one disadvantage with process-keyed tables is that you tend to put them in the same database as your other tables. This has two ramifications:

10. The tables are subject to complete logging; temp tables are only logged for rollbacks, not for recovery on start-up, since tempdb is always recreated when SQL Server starts.
11. If the database has full recovery, the process-keyed table will consume extra space in your transaction-log backups.

The second point can be addressed by putting all your process-keyed tables in a separate database with simple recovery. Both points can be addressed by using a global temp table, which I will discuss in the next session.

Using Memory-optimised Tables in SQL 2014

If you are on SQL 2014 – which of this writing is only available as a CTP and not yet released – there is an excellent solution to this problem. Use a table created with these options:

```
WITH (MEMORY_OPTIMIZED = ON, DURABILITY = SCHEMA_ONLY)
```

This is part of an entirely new feature in SQL 2014, known under the code name *Hekaton*. Hekaton tables are entirely in memory, and for really blazing performance you access them from stored procedures that have been compiled to C code. You can also access them from traditional T-SQL and still see significant performance improvements compared to traditional tables. Hekaton tables can hold persistent data, just like regular tables, and in such case there is still an overhead of writing to the transaction log. However, when you create a Hekaton table, there is also the option to say you don't want data to be durable. Such tables need very little logging, and this is perfect for process-keyed tables, where you don't want any data to survive a server crash.

A couple of notes:

12. The database must be configured to permit Hekaton tables. Specifically, you need to add a filegroup for memory-optimized data. (This is a directory, akin to what you have for FILESTREAM data.)
13. The surface area for Hekaton is limited, and currently these data types are *not* supported: **MAX** data types, **xml**, **text**, **ntext**, **image**, **sql_variant**, **datetimeoffset** and CLR data types. Furthermore, the maximum data size for a row must not exceed 8060 bytes.
14. You must define a primary key for the table, which must be either nonclustered or a hash (a new index type, specific to Hekaton). This can be a bit of a bummer – some of my process-keyed tables are a bit denormalised as they contain different type of data.
15. While I mentioned natively compiled stored procedures, these have a very limited feature set, and it is not likely that you will have much use for them with your process-keyed tables. For one thing, natively compiled stored procedures cannot access disk-based tables, which you probably want to join your process-keyed table with.
16. At this writing, it is not clear whether Hekaton will be available in all editions of SQL Server.

Conclusion

While process-keyed tables are not without issues when it comes to performance, and they are certainly a bit heavy-handed for the simpler cases, I still see this is the best overall solution that I present in this article. It does not come with a ton of restrictions like table-valued functions and it is *robust*, meaning that code will not break because of simple changes in difference to some of the other methods we will look at later.

But that does not mean that using a process-keyed table is always the way to go. For instance, if you only need output-only, and your procedure can be written as a table-valued function, that should be your choice.

Global Temp Tables

If you create a table with two leading hash marks (e.g. **##temp**), this is a global temp table. In difference to a regular temp table, a global temp table is visible to all processes. However, when the process that created the table goes away, so does the table (with some delay if another process is running a query against the table in that precise moment). That makes global temp tables difficult to use on any real global basis, but only when you have control over all involved processes like when spawning a second process through **xp_cmdshell**.

Nevertheless, there is a special case that SQL Server MVP Itzik Ben-Gan made me aware of: if you create a global temp table in a start-up procedure, the global temp table will be around as long as the server is up, unless someone explicitly drops it. This makes it possible to use a global temp table as a process-keyed table. This way you can have a fixed and known schema for your process-keyed table but still get the reduced logging of tempdb.

Here is a quick sample of how you create a global temp table when SQL Server starts:

```
USE master go CREATE PROCEDURE create_global_temp AS CREATE TABLE ##global(process_key  
uniqueidentifier NOT NULL, -- other columns here ) go EXEC sp_procoption create_global_temp,  
'startup', 'true'
```

It cannot be denied that there are some problems with this solution. What if you need to change the definition of the global temp table in way that cannot be handled with ALTER TABLE? Having to restart the server to get the new definition in place may not be acceptable. One way to address is to refer to your process-keyed table through a synonym (a feature added in SQL 2005). In development, you let the synonym point to a local table, and only when you are ready for production you change the synonym to refer to the global temp table. If you need to change the table definition while system is live, you create the new version of the table in the local database and change the synonym and run it that way until the server is restarted.

Table-valued Parameters

Table-valued parameters (TVP) were introduced in SQL 2008. They permit you to pass a table variable as a parameter to a stored procedure. When you create your procedure, you don't put the table definition directly in the parameter list, instead you first have to create a table type and use that in the procedure definition. At first glance, it may seem like an extra step of work, but when you think of it, it makes very much sense: you will need to declare the table in at least two places, in the caller and in the callee. So why not have the definition in one place?

Here is a quick example of a table-valued parameter in play:

```
CREATE TYPE my_table_type AS TABLE(a int NOT NULL, b int NOT NULL) go
CREATE PROCEDURE the_callee @indata my_table_type READONLY AS INSERT targettable (col1, col2) SELECT a, b FROM @indata go
CREATE PROCEDURE the_caller AS DECLARE @data my_table_type INSERT @data (a, b) VALUES (5, 7) EXEC the_callee @data go
```

One thing to note is that a table-valued parameter always has an implicit default value of an empty table. So saying EXEC the_callee in this example would not be an error.

Table-valued parameters certainly seem like the definite solution, don't they? Unfortunately, TVPs have a very limited usage for the problems I'm discussing in this article. If you look closely at the procedure definition, you find the keyword READONLY. And that is not an optional keyword, but it is compulsory for TVPs. So if you want to use TVPs to pass data between stored procedures, they are usable solely for input-only scenarios. I don't know about you, but in almost all situations where I [share a temp table](#) or use a [process-keyed table](#) it's for input-output or output-only.

When I first heard that SQL 2008 was to have TVPs, I was really excited. And when I learnt that they were readonly, I was equally disappointed. During the beta of SQL 2008 I wrote an article, [Why read-only table parameters is not enough](#), where I tried to whip up support for a [Connect item](#) in order to persuade the dev team to permit read-write TVPs when they are passed between stored procedures. The Connect item is still active, but with the release of SQL Server 2012 around the corner, the limitation is still there. Let's really hope that in the next version of SQL Server, we can use table parameters to pass data in all directions!

Note: While outside the scope for this article, table-valued parameters is still a welcome addition to SQL Server, since it makes it a lot easier to pass a set of data from client to server, and in this context the READONLY restriction is not a big deal. I give an introduction how to use TVPs from ADO .Net in my article [Arrays and Lists in SQL Server 2008](#).

INSERT-EXEC

Overview

INSERT-EXEC is a method that has been in the product for a long time. It's a method that is seemingly very appealing, because it's very simple to use and understand. Also, it permits you use the result of a stored procedure without any changes to it. Above we had the example with the procedure SalesByStore. Here is how we can implement BigSalesByStore with INSERT-EXEC:

```
CREATE PROCEDURE SalesByStore @storeid varchar(30) AS SELECT t.title, s.qty FROM sales s JOIN titles t ON t.title_id = s.title_id WHERE s.stor_id = @storeid go
CREATE PROCEDURE BigSalesByStore @storeid varchar(30), @qty smallint AS CREATE TABLE #SalesByStore(title varchar(80) NOT NULL PRIMARY KEY, qty smallint NOT NULL) INSERT #SalesByStore (title, qty) EXEC SalesByStore @storeid SELECT * FROM #SalesByStore WHERE qty >= @qty go
EXEC SalesByStore '7131' EXEC BigSalesByStore '7131', 25 go
DROP PROCEDURE SalesByStore, BigSalesByStore
```

In this example, I receive the data in a temp table, but it could also be a permanent table or a table variable. (Except on SQL 2000, where you cannot use a table variable.)

It cannot be denied that this solution is simpler than the solution with sharing a temp table. So why then did I first present a more complex solution? Because when we peel off the surface, we find that this method has a couple of issues that are quite problematic.

It Can't Nest

If you for some reason would try:

```
CREATE TABLE #BigSalesByStore(titleid varchar(80) NOT NULL PRIMARY KEY, qty smallint NOT NULL)
INSERT #BigSalesByStore (titleid, qty) EXEC BigSalesByStore '7131', 25
```

SQL Server will tell you:

Msg 8164, Level 16, State 1, Procedure BigSalesByStore, Line 8 An INSERT EXEC statement cannot be nested.

This is a restriction in SQL Server and there is not much you can do about it. Except than to save the use of INSERT-EXEC until when you really need it. That is, when rewriting the callee is out of the question, for instance because it is a system stored procedure.

There is a Serious Maintenance Problem

Six months later there is a user requirement for the application function that uses the result set from SalesByStore that the column **title_id** should be displayed. A developer merrily adds the column to the result set. Unfortunately, any attempt to use the function calling BigSalesByStore now ends in tears:

Msg 213, Level 16, State 7, Procedure SalesByStore, Line 2 Column name or number of supplied values does not match table definition.

What it says. The result set from the called procedure must match the column list in the INSERT statement exactly. The procedure may produce multiple result sets, and that's alright as long as all of them match the INSERT statement.

From my perspective, having spent a lot of my professional life with systems development, this is completely unacceptable. Yes, there are many ways to break code in SQL Server. For instance, a developer could add a new mandatory parameter to SalesByStore and that would also break BigSalesByStore. But most developers are aware the risks with such a change to an API and therefore adds a default value for the new parameter. Likewise, most developers understand that removing a column from a result set could break client code that expects that column and they would not do this without checking all code that uses the procedure. But *adding* a column to a result set seems so innocent. And what is really bad: there is no way to find out that there is a dependency – save searching through all the database code for calls.

Provided that you may alter the procedure you are calling, there are two ways to alleviate the problem. One is simply to add a comment in the code of the callee, so that the next developer that comes around is made aware of the dependency and hopefully changes your procedure as well.

Another way is to use table types (if you are on SQL 2008 or later). Here is an example:

```
CREATE TYPE SalesByStore_tbl AS TABLE (titleid varchar(80) NOT NULL PRIMARY KEY, qty smallint NOT NULL)
go CREATE PROCEDURE SalesByStore @storeid varchar(30) AS DECLARE @ret SalesByStore_tbl
INSERT @ret (titleid, qty) SELECT t.title, s.qty FROM sales s JOIN titles t ON t.title_id = s.title_id WHERE
```



```
s.stor_id = @storeid SELECT * FROM @ret go CREATE PROCEDURE BigSalesByStore @storeid
varchar(30), @qty smallint AS DECLARE @data SalesByStore_tbl INSERT @data EXEC SalesByStore
@storeid SELECT title, qty FROM @data WHERE qty >= @qty go EXEC SalesByStore '7131' EXEC
BigSalesByStore '7131', 25 go DROP PROCEDURE SalesByStore, BigSalesByStore DROP TYPE
SalesByStore_tbl
```

It is interesting to note that this code makes virtue of two things that usually are bad practice, to wit SELECT * and INSERT with out an explicit column list. This is not a matter of sloppiness – it is essential here. If someone wants to extend the result set of SalesByStore, the developer has to change the table type, and BigSalesByStore will survive, even if the developer does not know about its existence.

You could argue that this almost like an output TVP, but don't forget the other problems with INSERT-EXEC – of which there are two more to cover.

The Procedure is Executed in the Context of a Transaction

Even if there is no explicit transaction started with BEGIN TRANSACTION, an INSERT statement constitutes a transaction of its own. (So that the statement can be rolled back in case of an error.) That includes any procedure called through INSERT-EXEC. Is this bad or not? In many cases, this is not much of an issue. But there are a couple of situations where this can cause problems:

- The procedure performs an update intended to be quick. Locks are now held for a longer duration, which may cause contention problems.
- The isolation level is REPEATABLE READ or SERIALIZABLE, as opposed to the default READ COMMITTED. This also causes locks to held longer than intended.
- Some system procedures disagree to be called within a transaction.
- If the procedure accesses a linked server, you now have a distributed transaction. Distributed transactions are sometimes difficult to get working. See more about this in the closing chapter on [linked servers](#).

Rollback and Error Handling is Difficult

In my article on [error handling](#), I suggest that you should always have an error handler like

```
BEGIN CATCH IF @@trancount > 0 ROLLBACK TRANSACTION EXEC error_handler_sp RETURN 55555 END
CATCH
```

The idea is that even if you do not start a transaction in the procedure, you should always include a ROLLBACK, because if you were not able to fulfil your contract, the transaction is not valid.

Unfortunately, this does not work well with INSERT-EXEC. If the called procedure executes a ROLLBACK statement, this happens:

Msg 3915, Level 16, State 0, Procedure SalesByStore, Line 9 Cannot use the ROLLBACK statement within an INSERT-EXEC statement.

The execution of the stored procedure is aborted. If there is no CATCH handler anywhere, the entire batch is aborted, and the transaction is rolled back. If the INSERT-EXEC is inside TRY-CATCH, that CATCH handler will fire, but the transaction is doomed, that is, you must roll it back. The net effect is that the rollback is achieved as requested, but the original error message that triggered the rollback is lost. That

may seem like a small thing, but it makes troubleshooting much more difficult, because when you see this error, all you know is that something went wrong, but you don't know what.

And, no, before you ask, there is no way to find out at run-time that you are called from INSERT-EXEC.

Dynamic SQL

You can also use INSERT-EXEC with [dynamic SQL](#):

```
INSERT #tmp (...) EXEC sp_executesql @sql, @params, @par1, ...
```

Presumably, you have created the statement in **@sql** within your stored procedure, so it is unlikely that a change in the result set will go unnoticed. So from this perspective, INSERT-EXEC is fine. But the restriction that INSERT-EXEC can't nest remains, so if you use it, no one can call you with INSERT-EXEC. For this reason, in many cases it is better to put the INSERT statement inside the dynamic SQL.

There is also a performance aspect, that SQL Server MVP Adam Machanic has detailed in a [blog post](#). The short summary is that with INSERT-EXEC, data does not go directly to the target table but bounces over a "parameter table", which incurs some overhead. Then again, if your target table is a temp table, and you put the INSERT inside the dynamic SQL, you may face a performance issue because of recompilation.

Occasionally, I see people who use INSERT-EXEC to get back scalar values from their dynamic SQL statement, which they typically invoke with EXEC(). In this case, you should not use INSERT-EXEC at all, but instead use [sp_executesql](#) which permits you to use OUTPUT parameters. Dynamic SQL is a complex topic, and if you are not acquainted with it, I recommend you to read my article [The Curse and Blessings of Dynamic SQL](#).

Conclusion

INSERT-EXEC is simple to use, and if all you want to do is to grab a big result set from a stored procedure for further analysis ad hoc, it's alright.

But you should be very restrictive to use it in application code. Only use it when rewriting the procedure you are calling is completely out of the question. That is, the procedure is not part of your application: a system stored procedure or part of a third-party product. And in this case, you should make it a routine to always test your code before you take a new version of the other product in use.

Using the CLR

If INSERT-EXEC shines in its simplicity, using the CLR is complex and bulky. It is not likely to be your first choice, and nor should it. However, if you are in the situation that you cannot change the callee, and nor it possible for you to use [INSERT-EXEC](#) because of any of its limitations, the CLR can be your last resort.

As a recap, here are the main situations where [INSERT-EXEC](#) fails you, and you would want to turn to the CLR:

- The called procedure returns several result sets with different structures. This is true for many system procedures in SQL Server.
- The called procedure cannot be called within an active transaction. See the final note in this section for an example.
- The called procedure already uses [INSERT-EXEC](#).

- The called procedure accesses a linked server, and you cannot get the distributed transaction to work.

The CLR has one more advantage over INSERT-EXEC: it is less sensitive to changes in the procedure you call. If a column is added to the result set of the procedure, your CLR procedure will not break.

The idea as such is simple: you write a stored procedure in a CLR language like C# or VB .NET that runs the callee and captures the result set(s) into a **DataSet** object. Then you write the data from the DataSet back to the table where you want the data. While simple, you need to write some code.

Let's have a look at an example. When you call the system procedure **sp_helpdb** for a specific database, it produces two result sets, of which the second result set lists the files for the database. Say that you want to gather this output for all databases on the server. You cannot use [INSERT-EXEC](#) due to the multiple result sets. To address this issue, I wrote a stored procedure in C# that you find in the file [helpdb.cs](#). In the script [helpdb.sql](#) you can see how I employ it. The C# procedure first runs **sp_helpdb** with the **DataAdapter.Fill** method to get the data into a DataSet. It then iterates over the rows in the second **DataTable** in the DataSet and inserts these into the temp table created by the SQL script.

On SQL 2008 it is possible to simplify the solution somewhat with help of a [table-valued parameter](#). I've written a second version of the **helpdb** procedure, available in the file [helpdb-2008.cs](#), where I simply pass the DataTable to the INSERT statement with a TVP and insert all rows in one go. To achieve this, I need to create a table type. I like to highlight two more things in **helpdb-2008**:

- Since I want the database name in the final output, I pass this as a separate parameter to the INSERT statement, as it is not included in the DataTable.
- The first column from **sp_helpdb** is called **name**, and in the temp table I've changed that to **logicalname** to make the final output clearer. However, in the table type the column is called **name**, since it must match the names in the DataTable which gets its names from the output of **sp_helpdb**.

Undoubtedly, this solution requires more work. You need to write more code than with most other methods, and you get an assembly that you must somehow deploy. If you already are using the CLR in your database, you probably already have routines for dealing with assemblies. But if you are not, that first assembly you add to the database is quite of a step to take. A further complication is that the CLR in SQL Server is disabled by default. To enable it, you (or the DBA) need to run:

```
EXEC sp_configure 'clr enabled', 1 RECONFIGURE
```

Another issue is that this solution goes against best practices for using the CLR in SQL Server. First of all, data access from the CLR should be avoided, simply because T-SQL is better equipped for this. But here we are talking about situations where we need to circumvent limitations in T-SQL. Another violation of best practice is the use of the DataAdapter, DataTable and DataSet classes. This is something to be avoided, because it means that you have data in memory in SQL Server outside the buffer pool. Of course, a few megabytes is not an issue, but if you would read several gigabytes of data into a DataSet, this could have quite nasty effects for the stability of the entire SQL Server process.

The alternative is to use a plain **ExecuteReader** and insert the rows as they come, possibly buffering them in small sets of say 500 rows to improve performance. This is certainly a viable solution, but it makes deployment even more difficult. To wit, you cannot perform the INSERT statements on the context connection while the DataReader is running, so you would need to open a second connection and this

requires that the assembly has the permission `EXTERNAL_ACCESS`. So for practical purposes, you would only go this road, if you are anxious that you will read too much data than what is defensible for a `DataSet`.

Note: Initially, I got this idea when SQL Server MVP Steve Jones tried to run `DBCC SHRINKDATABASE` from `INSERT-EXEC` to capture the result set that `SHRINKDATABASE` produces. However, this command cannot be run inside a transaction, so that did not work out. I suggested to him that the CLR could work, and when I tested it I found that it did ... on SQL 2008 only. On SQL 2005, my process was killed with an access violation (which means a bug in SQL Server), so in this particular case not even the last resort worked.

OPENQUERY

Introduction

Just like [INSERT-EXEC](#) this is a method where you can use the called stored procedure as-is. The purpose of `OPENQUERY` and its cousin `OPENROWSET` is to permit you to run pass-through queries on linked servers. It can be very useful, not the least if you want to join multiple tables on the remote server and want to be sure that the join is evaluated remotely. Instead of accessing a remote server, you can make a loopback connection to your own server, so you can to say things like:

```
SELECT * FROM OPENQUERY(LOCALSERVER, 'EXEC sp_who') WHERE status = 'runnable'
```

If you want to create a table from the output of a stored procedure with `SELECT INTO` to save typing, this is the only method in the article that fits the bill.

So far, `OPENQUERY` looks very simple, but as this chapter moves on you will learn that `OPENQUERY` can be very difficult to use. Moreover, it is not aimed at improving performance. It may save you from rewriting your stored procedure, but most likely you will have to put in more work overall – and in the end you get a poorer solution. While I'm not enthusiastic over `INSERT-EXEC`, it is still a far better choice than `OPENQUERY`.

Setup

In the example, `LOCALSERVER` may look like a keyword, but it is only name. This is how you define it:

```
EXEC sp_addlinkedserver @server = 'LOCALSERVER', @srvproduct = '', @provider = 'SQLOLEDB',  
@datasrc = @@servername
```

To create a linked server, you must have the permission `ALTER ANY SERVER`, or be a member of any of the fixed server roles **sysadmin** or **setupadmin**. Instead of `SQLOLEDB`, you can specify `SQLNCLI`, `SQLNCLI10` or `SQLNCLI11` depending on your version of SQL Server. SQL Server seems to use the most recent version of the provider anyway.

Implications of Using a Loopback Connection

It's important to understand that `OPENQUERY` opens a new connection to SQL Server. This has some implications:

- The procedure that you call with `OPENQUERY` cannot refer temp tables created in the current connection.
- The new connection has its own default database (defined with `sp_addlinkedserver`, default is *master*), so all object specifications must include a database name.

- If you have an open transaction and you are holding locks when you call OPENQUERY, the called procedure can not access what you lock. That is, if you are not careful you will block yourself.
- Connecting is not for free, so there is a performance penalty.
- There is also a performance penalty for passing the data out from SQL Server and back. Even if there is no network involved, data is copied twice extra compared to a plain SELECT query. This can be costly if the result set is big.

ANSI Settings

The settings ANSI_NULLS and ANSI_WARNINGS must be ON for queries involving linked servers. Thankfully, these settings are also on by default in most contexts. There are mainly two exceptions: 1) very old client APIs like DB-Library. 2) If you are still on SQL 2000, beware that if you create stored procedures from Enterprise Manager, they will be created with ANSI_NULLS OFF, and this is a setting that is saved with the procedure and overrides the setting for the connection. (If you use Query Analyzer to create your procedures, the issue does not arise.) It's not very likely that you will run into this issue, but if you see the error message

Msg 7405, Level 16, State 1, Line 1 Heterogeneous queries require the ANSI_NULLS and ANSI_WARNINGS options to be set for the connection. This ensures consistent query semantics. Enable these options and then reissue your query.

you will need to investigate where the bad setting is coming from.

The Query Parameter

The second parameter to OPENQUERY is the query to run on the remote server, and you may expect to be able to use a variable here, but you cannot. The query string must be a constant, since SQL Server needs to be able to determine the shape of the result set at compile time. This means that you as soon your query has a parameter value, you need to use dynamic SQL. Here is how to implement BigSalesByStore with OPENQUERY:

```
CREATE FUNCTION quotestring(@str nvarchar(MAX)) RETURNS nvarchar(MAX) AS BEGIN DECLARE @ret
nvarchar(MAX), @sq char(1) SELECT @sq = '' SELECT @ret = replace(@str, @sq, @sq + @sq)
RETURN(@sq + @ret + @sq) END go CREATE PROCEDURE SalesByStore @storeid varchar(30) AS SELECT
t.title, s.qty FROM sales s JOIN titles t ON t.title_id = s.title_id WHERE s.stor_id = @storeid go CREATE
PROCEDURE BigSalesByStore @storeid varchar(30), @qty smallint, @debug bit = 0 AS DECLARE
@remotesql nvarchar(MAX), @localsql nvarchar(MAX) SELECT @remotesql = 'EXEC ' +
quotename(db_name()) + '.dbo.SalesByStore ' + dbo.quotestring(@storeid) SELECT @localsql = 'SELECT *
FROM OPENQUERY(LOCALSERVER, ' + dbo.quotestring(@remotesql) + ') WHERE qty >= @qty' IF @debug
= 1 PRINT @localsql EXEC sp_executesql @localsql, N'@qty smallint', @qty go EXEC SalesByStore '7131'
EXEC BigSalesByStore '7131', 25, 1 go DROP PROCEDURE BigSalesByStore, SalesByStore DROP FUNCTION
quotestring
```

What initially seemed simple to use, is no longer so simple. What I did not say above is there are *two* reasons why we need dynamic SQL here. Beside the parameter **@storeid**, there is also the database name. Since OPENQUERY opens a loopback connection, the EXEC statement must include the database name. Yes, you could hardcode the name, but sooner or later that will bite you, if nothing else the day you want to restore a copy of your database on the same server for test purposes. From this follows that in practice,

there are not many situations in application code where you can use OPENQUERY without having to use dynamic SQL.

The code certainly requires some explanation. The function **quotestring** is a helper, taken from my article on [dynamic SQL](#). It encloses a string in single quotes, and doubles any quotes within it to conform to the T-SQL syntax. The problem with writing dynamic SQL which involves OPENQUERY is that you get at least three levels of nested strings, and if you try to do all at once, you will find yourself writing code which has up to eight consecutive single quotes that you or no one else can read. Therefore it is *essential* to approach the problem in a structured way like I do above. I first form the query on the remote server, and I use quotestring to embed the store id. Then I form the SQL string to execute locally, and again I use quotestring to embed the remote query. I could also have embedded **@qty** in the string, but I prefer to adhere to best practices and pass it as a parameter to the dynamic SQL string. As always when I use dynamic SQL, I include a **@debug** parameter, so that I can inspect the statement I've generated.

Note: the example is written for SQL 2005 and later. If you are on SQL 2000, you need to replace all occurrences of MAX with 4000.

The Battle with FMTONLY ON

To be able to compile a query that includes OPENQUERY, SQL Server must retrieve metadata from the linked server to determine the shape of the result set for the pass-through query. SQL Server makes all connections to the linked server through OLE DB, and the way OLE DB determines metadata on SQL Server up to SQL 2008 is to run the command batch preceded by SET FMTONLY ON. When FMTONLY is ON, SQL Server does not execute any data-retrieving statements, but only sifts through the statements to return metadata about the result sets. FMTONLY can be a source for confusion in more than one way. One situation is a procedure that creates a temp table: you will get an error message, since the table never gets created in FMTONLY mode. Here is one example:

```
SELECT * FROM OPENQUERY(LOCALSERVER, 'EXEC msdb..sp_helpindex sysjobs')
```

On SQL 2008, this results in:

Msg 208, Level 16, State 1, Procedure sp_helpindex, Line 104 Invalid object name '#spindtab'.

(The actual error message is different on about every version of SQL Server.)

This happens because the CREATE TABLE statement for the temp table is not executed. (Note that this is different for table variables. Since they are declared entities, they exist from the moment the procedure starts executing, FMTONLY or not.) Now when we know why this error occurs, we can spot a workaround:

```
SELECT * FROM OPENQUERY(LOCALSERVER, 'SET FMTONLY OFF EXEC msdb..sp_helpindex sysjobs')
```

That is, we override the FMTONLY ON setting. But beware! This means that the procedure is executed twice, so there certainly is a performance cost. Moreover, if the procedure performs updating actions, these are also performed twice which is likely to be the completely wrong thing to do. While I mention this trick here, I strongly recommend against using it, particularly in production code. This becomes even more emphasised with the release of SQL Server 2012: on SQL 2012, SET FMTONLY OFF has no effect at all! I will come back to why and what the alternatives are.

If you absolutely want to use a stored procedure that uses a temp table, here is a different trick. This definitely counts as one of the most obscure pieces of T-SQL I've ever come up with:

```
CREATE PROCEDURE temp_temp_trick AS DECLARE @fmtonlyon int SELECT @fmtonlyon = 0 IF 1 = 0  
SELECT @fmtonlyon = 1 SET FMTONLY OFF CREATE TABLE #temp(...) IF @fmtonlyon = 1 SET FMTONLY  
ON -- Rest of the code goes here.
```

The reason that this works is that when FMTONLY is in effect, IF conditions are not evaluated, but both branches of IF ELSE are "executed". And while queries and CREATE TABLE statements are not performed in FMTONLY mode, variable assignments are. This way we lure SQL Server to create the temp table at compile-time, but the full procedure is not executed. This trick is arguably better than putting SET FMTONLY OFF in the call to OPENQUERY. But it still does not work with SQL 2012, so it is something you should put in code that is supposed to live for a couple of years. And obviously, this is not an option, if you cannot change the procedure you are calling.

Another common reason you get problems with FMTONLY is that the result set is produced in dynamic SQL. Again, this prevents SQL Server from determining the metadata. The famous, but undocumented, system stored procedure **sp_who2** uses dynamic SQL to size the columns of the result set. On SQL 2008 the query

```
SELECT * FROM OPENQUERY(LOCALSERVER, 'EXEC sp_who2')
```

results in:

Msg 7357, Level 16, State 2, Line 1 Cannot process the object "EXEC sp_who2". The OLE DB provider "SQLNCLI10" for linked server "LOCALSERVER" indicates that either the object has no columns or the current user does not have permissions on that object.

Again, the workaround with SET FMTONLY OFF can be applied if you are on SQL 2008 or earlier. (Except that in this particular example it still does not work on SQL 2005 and SQL 2008 because **sp_who2** returns two columns called SPID.) The same caveats apply: the procedure is executed twice, and it does not work on SQL 2012. So don't go there.

Metadata Retrieval in SQL 2012

In SQL 2012, Microsoft have scrapped SET FMTONLY ON which never did a good job; there are several other issues with it, that I have not covered here. Instead they use the new stored procedure **sp_describe_first_result_set** which is a more robust way to determine metadata, why the trick with SET FMTONLY OFF is no longer applicable. (To clarify: SET FMTONLY ON still works in SQL 2012 to support calls from legacy clients, and SQL 2012 also uses SET FMTONLY ON for linked servers running earlier versions of SQL Server. But for a loopback connection, SQL 2012 only uses **sp_describe_first_result_set**.)

While this procedure avoids many of the problems with SET FMTONLY ON, you will still get an error if your procedure uses a temp table or dynamic SQL. The good news is that Microsoft now offers a method to describe the result set. You can say:

```
SELECT * FROM OPENQUERY(LOCALSERVER, 'EXEC msdb..sp_helpindex sysjobs WITH RESULT SETS  
((index_name sysname, index_description nvarchar(500), index_keys nvarchar(500)))')
```


That is, the new WITH RESULT SETS clause permits you to declare how the result set from the stored procedure looks like. SQL Server validates the actual result set against the description, and if there is an inconsistency, you will get an error. Pay attention to the syntax: the column list is enclosed in *two* pair of parentheses.

Rather than specify a list of columns, you can also specify the name of a table, view, table-valued function or a table type. Please refer to the topic on EXECUTE in Books Online for full details on the WITH RESULT SETS clause.

If you want to know how the result set from the stored procedure you are calling looks like, you can say:

```
EXEC sp_describe_first_result_set N'your_sp'
```

It has to be admitted that it takes some work to translate the output to a column list for WITH RESULT SETS. And obviously, if you wanted to use SELECT INTO with OPENQUERY to save your from typing a CREATE TABLE statement, this was not what you wanted.

The Effect of DML Statements

Yet a problem with OPENQUERY is demonstrated by this script:

```
CREATE TABLE nisse (a int NOT NULL) go CREATE PROCEDURE silly_sp @x int AS --SET NOCOUNT ON
INSERT nisse VALUES (@x) SELECT @x * @@trancount SELECT @x * 3 go SELECT * FROM
OPENQUERY(LOCALSERVER, 'EXEC tempdb.dbo.silly_sp 7') go SELECT * FROM nisse go
```

The script yields the same errors as for **sp_who2**, saying that there are no columns. The reason for this message is that the first "result set" is the *rows affected* message generated by the INSERT statement, and this message lures OPENQUERY to think that there were no columns in the result set. Adding SET NOCOUNT ON to the procedure resolves this issue. You could also add SET NOCOUNT ON the command string you pass to OPENQUERY. (And in difference to tricking with SET FMTONLY ON, this is a perfectly valid thing to do.)

Implicit Transactions

When SQL Server executes the query for real, the OLE DB provider first issues SET IMPLICIT_TRANSACTIONS ON. With this setting SQL Server starts a transaction when an INSERT, UPDATE or DELETE statement is executed. (This also applies to a few more statements, see Books Online for details.) This can give some surprises. For instance, take the script above. Once SET NOCOUNT ON is in force, this is the output:

```
----- 7 (1 row(s) affected) a ----- (0 row(s) affected)
```

We get back '7' from the call to **silly_sp**, which indicates that **@@trancount** is 1, and there is thus an open transaction, despite there is no BEGIN TRANSACTION in the procedure. (We don't get the '21' that we get when we execute **silly_sp** directly, because with OPENQUERY, we only get one result set.) You also see that when we SELECT directly from **nisse** after the call to OPENQUERY, that the table is empty. This is because the implicit transaction was rolled back.

Final Words

As you have seen, at first OPENQUERY seems very simple to use, but the stakes quickly gets higher. If you are still considering to use OPENQUERY after having read this section, I can only wish you good luck

and I hope that you really understand what you are doing. OPENQUERY was not intended for accessing the local server, and you should think twice before you use it that way.

Using XML

Introduction

XML is a solution that aims at the same spot as [sharing a temp table](#) and [process-keyed tables](#). That is, the realm of general solutions without restrictions, to the price of a little more work. While SQL 2000 has support for XML, if you want to use XML to pass data between stored procedures, you need to have at least SQL 2005.

Constructing the XML

We will look at a version of SalesByStore and BigSalesByStore which uses XML, but since this is a little too much to digest in one go, we first only look at SalesByStore_core to see how we construct the XML:

```
CREATE PROCEDURE SalesByStore_core @storeid varchar(30), @xmldata xml OUTPUT AS SET @xmldata = ( SELECT t.title, s.qty FROM sales s JOIN titles t ON t.title_id = s.title_id WHERE s.stor_id = @storeid FOR XML RAW('SalesByStore'), TYPE) go
```

In the previous version of SalesByStore_core, we stored the data from the result in a temp table. Here we use FOR XML RAW to generate an XML document that we save to the output parameter **@xmldata**.

This is how the resulting XML document may look like:

```
<SalesByStore title="Is Anger the Enemy?" qty="20" /> <SalesByStore title="The Gourmet Microwave" qty="25" /> <SalesByStore title="Computer Phobic AND Non-Phobic Individuals: Behavior Variations" qty="20" /> <SalesByStore title="Life Without Fear" qty="25" /> <SalesByStore title="Prolonged Data Deprivation: Four Case Studies" qty="15" /> <SalesByStore title="Emotional Security: A New Algorithm" qty="25" />
```

FOR XML has three more options beside RAW: AUTO, ELEMENTS and PATH, but for our purposes here, RAW is the simplest to use. You don't have to specify a name for the elements; the default in this case will be **row**, but I would suggest that using a name is good for clarity.

The keyword TYPE ensures that the return type of the SELECT query is the **xml** data type; without TYPE the type would be **nvarchar(MAX)**. TYPE is not needed here, since there will be an implicit conversion to **xml** anyway, but it can be considered good practice to include it. Except... there is a [bug](#) in SQL 2008 which makes XML documents created with TYPE to be less efficient.

Converting the XML Data Back to Tabular Format

Since SalesByStore should work like it did originally, it has to convert the data back to tabular format, a process known as *shredding*. Here is how the XML version looks like:

```
CREATE PROCEDURE SalesByStore @storeid varchar(30) AS DECLARE @xmldata xml EXEC SalesByStore_core @storeid, @xmldata OUTPUT SELECT T.c.value('@title', 'varchar(80)') AS title, T.c.value('@qty', 'smallint') AS qty FROM @xmldata.nodes('SalesByStore') AS T(c) go
```

To shred the document, we use two of the **xml** type methods. The first is **nodes** which shreds the documents into fragments of a single element. That is, this part:

```
FROM @xmldata.nodes('SalesByStore') AS T(c)
```

The part **T(c)** defines as alias for the one-column table as well as an alias for the column. To get the values out of the fragments, we use another **xml** type method, **value**, to get the individual values out of the fragment. The **value** method takes two arguments whereof the first addresses the value we want to extract, and the second specifies the data type. The first parameter is a fairly complex story, but as long as you follow the example above, you don't really need to know any more. Just keep in mind that you must put an **@** before the attribute names, else you would be addressing an element. In the [XML](#) section of my article *Arrays and Lists in SQL Server 2005 and Beyond*, I have some more information about **nodes** and **value**.

To make the example complete, here is the XML version of BigSalesByStore. To avoid having to repeat the call to **value** in the WHERE clause, I use a CTE (Common Table Expression).

```
CREATE PROCEDURE BigSalesByStore @storeid varchar(30), @qty smallint AS DECLARE @xmldata xml
EXEC SalesByStore_core @storeid, @xmldata OUTPUT ; WITH SalesByStore AS ( SELECT
T.c.value('@title', 'varchar(80)') AS title, T.c.value('@qty', 'smallint') AS qty FROM
@xmldata.nodes('SalesByStore') AS T(c) ) SELECT title, qty FROM SalesByStore WHERE qty >= @qty go
```

Input and Output

In this is example the XML document is output-only, but it's easy to see that the same method can be used for input-only scenarios. The caller builds the XML document and the callee shreds it back to a table.

What about input-output scenarios like the procedure [ComputeTotalStoreQty](#)? One possibility is of course that the callee shreds the data into a temp table, performs its operation, and converts the data back to XML. A second alternative is that the callee modifies the XML directly using the **xml** type method **modify**. I will spare you from an example of this, however, as it unlikely that you would try it, unless you already are proficient in XQuery. A better alternative may be to mix methods: use a table-valued parameter for input and only use XML for output.

Parent-child Data

The result set in the example is from a single table, but what if we have some form of parent/child-relationship? Say that we want to return the name of all authors, as well as all the titles they have written. With [temp tables](#) or [process-keyed tables](#), the natural solution would be to use two tables (or actually three, since in *pubs* there is a many-to-many relationship between titles and authors, but I overlook this here.) But since XML is hierarchical, it would be more natural to put everything in a single XML document, and here is a query to do this:

```
SELECT a.au_id , a.au_lname, a.au_fname , (SELECT t.title FROM pubs..titleauthor ta JOIN pubs..titles t
ON t.title_id = ta.title_id WHERE a.au_id = ta.au_id FOR XML RAW('titles'), TYPE) FROM pubs..authors a
FOR XML RAW('authors'), TYPE
```

Rather than a regular join query, I use a subquery for the titles, because I only want one node per author with all titles. With a join, I get one author node for each title, so that authors with many books appear in multiple nodes. The subquery uses FOR XML to create a nested XML document, and this time the TYPE option is mandatory, since without it the nested XML data would be included as a plain string.

To retrieve the titles from the XML document, you could use this query:

```
SELECT au_id = A.item.value('@au_id', 'varchar(11)'), title = T.item.value('@title', 'varchar(80)') FROM
@x.nodes('/authors') AS A(item) CROSS APPLY A.item.nodes('titles') AS T(item)
```

The first call to **nodes** gives you a fragment per **authors** node, and then you use CROSS APPLY to dig down to the **titles** node. For a little longer discussion on this way of shredding a hierarchical XML document, see the [XML](#) section of my article *Arrays and Lists in SQL Server 2005 and Beyond*.

Assessing the Method

So far the technique to use this method. Let's now assess it. If you have never worked with XML in SQL Server, you are probably saying to yourself *I will never use **that!***. And one can hardly blame you. This method is like pushing the table camel through the needles eye of the parameter list of a stored procedure. Personally, I think the method spells k-l-u-d-g-e. But it's certainly a matter of opinion. I got a mail from David Walker, and he went as far as saying this is the *only* method that really works.

And, that cannot be denied, there are certainly advantages with XML over about all the other methods I have presented here. It is less contrived than using the [CLR](#), and it is definitely a better option than [OPENQUERY](#). You are not caught up with the limitations of [table-valued functions](#). Nor do you have any of the issues with [INSERT-EXEC](#). Compared to [temp tables](#) and [process-keyed tables](#), you don't have to be worried about recompilation or that programmers fail to clean up a process-keyed table after use.

When it comes to performance, you get some cost for building the XML document and shredding it shortly thereafter. Then again, as long as the amount of data is small, say less than 200 KB, the data will stay in memory and there is no logging involved like when you use a table of any sort. Larger XML documents will spill to disk, though. A general caveat is that inappropriate addressing in a large XML document can be a real performance killer, so if you expect large amounts of data, you have to be careful. (And these issues can appear with sizes below 200 KB.)

Besides the daunting complexity, there are downsides with XML from a robustness perspective. XML is more sensitive to errors. If you make a spelling mistake in the first argument to **value**, you will silently get NULL back, and no error message. Likewise, if you get the argument to **nodes** wrong, you will simply get no rows back. The same problem arises if you change a column alias or a node name in the FOR XML query, and forget to update a caller. When you use a process-keyed table or a temp table you will get an error message at some point, either at compile-time or at run-time.

Another weak point is that you have to specify the data type for each column in the call to **value**, inviting you to make the mistake to use different data types for the same value in different procedures. This mistake is certainly possible when use temp tables as well, although copy-and-paste are easier to apply on the latter. With a process-keyed table it cannot happen at all.

One thing I like with tables is that they give you a description of the data you are passing around; this is not the least important when many procedures are using the same process-keyed table. This is more difficult to achieve with XML. You could use schema collections for the task, but you will not find very many SQL Server DBAs who speak XSD fluently. Also, schema-bound XML tends to incur a performance penalty in SQL Server.

For these reasons, I feel that using a temp table or a process-keyed table are better choices than XML. And while I find XML an overall better method than INSERT-EXEC or OPENQUERY, these methods have the

advantage that you don't have to change the callee. So that kind of leaves XML in nowhere land. But as they say, your mileage may vary. If you feel that XML is your thing, go for it!

Using Cursor Variables

This method was suggested to me by Peter Radocchia. Cursor variables were introduced in SQL 7, but I suspect that many SQL developers are at most only dimly aware of their existence. I never use them myself. Here is an example of how you use them to bring the result set from one procedure to another:

```
CREATE PROCEDURE get_cursor @cursor CURSOR VARYING OUTPUT AS SET @cursor = CURSOR STATIC
FOR SELECT au_id, au_lname, au_fname FROM pubs..authors OPEN @cursor go CREATE PROCEDURE
caller AS DECLARE @cursor CURSOR DECLARE @au_id char(11), @au_fname varchar(40), @au_lname
varchar(40) SET NOCOUNT ON EXEC get_cursor @cursor OUTPUT WHILE 1 = 1 BEGIN FETCH NEXT FROM
@cursor into @au_id, @au_lname, @au_fname IF @@fetch_status <> 0 BREAK PRINT 'Au_id: ' +
@au_id + ', name: ' + @au_fname + ' ' + @au_lname END DEALLOCATE @cursor go EXEC caller go DROP
PROCEDURE caller, get_cursor
```

Note that the cursor is STATIC. Static cursors are much preferable over dynamic cursors, the default cursor type, since the latter essentially evaluates the query for every FETCH. When you use a static cursor, the result set of the SELECT statement is saved into a temp table, from where FETCH retrieves the data.

I will have to admit that I see little reason to use this method. Just like INSERT-EXEC, this method requires an exact match between the caller and the callee for the column list. And since data is processed row by row, performance is likely to take a serious toll if there are any volumes.

The Challenges of Linked Servers

If your procedures are on different servers, the level of difficulty rises steeply. There are many restrictions with linked servers, and several of the methods I have presented cannot be used at all. Ironically, some of the methods that I have discouraged you from, suddenly step up as the better alternatives. One reason for this is that with linked servers, things are difficult anyway.

It is somewhat easier to retrieve data from a procedure on a linked server than passing data to it, so let's look at output first. If you have an input-output scenario, you should probably look into mixing methods.

Output

If all you want to do is to get data back, these methods work:

[OUTPUT parameters](#) – but only for data types that are 8000 bytes or less. That is, you cannot retrieve the value of output parameters that are **varchar(MAX)** etc.

[INSERT-EXEC](#) – INSERT-EXEC works fine with linked servers. Actually even better than with local procedures, since if the procedure you call uses INSERT-EXEC, this will not matter. The only restriction is that the result set must not include types that are not supported in distributed queries, for instance **xml**. (**(n)varchar(MAX)** is OK.) The fact that INSERT-EXEC starts a transaction can cause a real nightmare, since the transaction now will be a distributed transaction and this requires that you configure MSDTC (Microsoft Distributed Transaction Coordinator) correctly. If both servers are in the same domain, it often works out of the box. If they are not, for instance because you only have a workgroup, it may be impossible (at least I have not been able to). On SQL 2008 and later, you may be able to escape the problem by setting the option **remote proc transaction promotion** for the linked server to **false**. (Note

that this affects all uses of the linked server, and there may be situations where a distributed transaction is desirable.)

[OPENQUERY](#) – since OPENQUERY is a feature for linked servers in the first place, there is no difference to what I discussed above. It is still difficult with lots of pitfalls, but the land of linked servers is overall difficult. Nevertheless, INSERT-EXEC will in many cases be simpler to use. But with OPENQUERY you don't have to bounce the remote data over a table, and if result set of the remote procedure is extended with more columns, your code will not break.

[Using the CLR](#) – Using the CLR for linked servers is interesting, because the normal step would be to connect to the remote server directly, and bypass the local definition of linked servers – and thereby bypass all restrictions with regards to data types. When you make a connection to a remote server through the CLR, the default is enlist into the current transaction, which means that you have to battle MSDTC. However, you can easily escape this battle by adding **enlist=false** in the connection string to the remote server. This works on all versions of SQL Server from SQL 2005 and on. When using the CLR to access a remote server, there are no obstacles with using **ExecuteReader** and store the data into a local table as they come, since you are using two different connections. For a CLR procedure to be able to access a remote server, the assembly must be installed with the permission **EXTERNAL_ACCESS**.

[XML](#) – You cannot use the **xml** data type in a call to a remote stored procedure. However, you can make the OUTPUT parameter to be **varchar(8000)** and return the XML document that way – if it fits.

The other methods do not work, and that includes user-defined functions. You cannot call a user-defined function on a linked server.

Input

If you want to pass a large amount of data for input over a linked server, there are three possibilities. Or three kludges if you like.

[XML](#) might be the easiest. The **xml** data type is not supported in calls to remote procedures, so you need to convert the XML document to **nvarchar(MAX)** or **varbinary(MAX)**. The parameter on the other side can still be **xml**.

You cannot pass a table-valued parameter to a remote stored procedure. But you could have a CLR stored procedure that connects to the remote server and passes the TVP directly; as noted above, the assembly needs to have the permission **EXTERNAL_ACCESS**. You cannot pass a TVP to a CLR stored procedure from T-SQL, so you would either have to pass the data as XML to the CLR procedure, or the CLR procedure would have read the data from a (temp) table.

The last alternative is really messy. The caller stores the data in a [process-keyed table](#) locally and then calls the remote procedure, passing the process-key. The remote procedure then calls back to the first server and either selects directly from the process-keyed table, or calls a procedure on the source server with [INSERT-EXEC](#). For an input-output scenario, the callee could write data back directly to the process-keyed table.

Acknowledgments and Feedback

The issue about using SET FMTONLY ON is something that I learnt from Umachandar Jayachandran at Microsoft. SQL Server MVP Tony Rogerson pointed out that a process-keyed table should have a clustered index on the process key. Simon Hayes suggested some clarifications. Peter Radocchia

suggested the [cursor](#) method. Richard St-Aubin and Wayne Bloss both suggested interesting approaches when [sharing temp tables](#). Thanks to SQL Server MVP Iztik Ben-Gan for making me aware of global temp tables and start-up procedures. Sankar Reddy pointed out to me that my original suggestion for XML as a solution for linked servers was flawed. Greg Borota pointed out that an old leftover from SQL 2000 still was in the text. SQL Server MVP Adam Machanic made some interesting revelations about INSERT-EXEC with dynamic SQL. David Walker encouraged me to write more in depth on XML, and SQL Server MVP Denis Gobo gave me a tip on that part. Jay Michael pointed out an error in the section on table parameters.

If you have suggestions for improvements, corrections on topic, language or formatting, please mail me at esquel@sommarskog.se. If you have technical questions that any knowledgeable person could answer, I encourage you to post to the [Transact-SQL forum](#) on MSDN/TechNet or any other SQL forum you frequent.

Revision History

2013-11-02 – Added a subsection about process-keyed tables how they could be implemented with [non-durable Hekaton tables](#) in SQL 2014.

2013-03-24 – Modified the subsection [A Maintenance Problem](#) to include a good suggestion from Wayne Bloss about using a table type as the base for a shared temp table.

2012-07-18 – There were a few errors and mumblings in the paragraph about using the CLR for [linked servers](#) that I have corrected. Particularly I incorrectly said that you would not be enlisted in any transaction unless you specify this. The reverse applies: by default you are enlisted, but you can include `enlist=true` to the connection string.

2012-05-11 – Added a [note](#) about SQL Server Data Tools (SSDT) and sharing temp tables.

2011-12-31 – I have performed a general overhaul of the article in hope to make things clearer. Particularly, there are now more complete examples for the various techniques. In terms of new technical content I have updated the article for SQL 2012, which mainly affects [OPENQUERY](#), since the trick with SET FMTONLY OFF does not work on SQL 2012. I have also expanded the closing chapter on [linked servers](#) a bit.

2010-01-10 – Extended the [XML](#) section with more examples and a deeper discussions on pros and cons. Updated the section [table parameters](#) for the fact that SQL 2008 is no longer in beta, and fixed error in code sample. Modified the section on [OPENQUERY](#) to explain why FMTONLY ON exists more accurately.

2009-06-29 – Added a brief discussion on performance about INSERT-EXEC with dynamic SQL, and a reference to a [blog post](#) from SQL Server MVP Adam Machanic.

2009-05-18 – The section on INSERT-EXEC said that it does not work with table variables, which is right on SQL 2000 only.

2008-08-16 – Added a trick for [sharing temp tables](#), suggested by Richard St-Aubin.

2008-06-06 – Added a section on [linked servers](#), and removed the note on linked servers in the [XML](#) section, since it was not very accurate.

2008-03-03 – Added a section on how could use the [CLR](#) when [INSERT-EXEC](#) fails you. Reviewed the section on [XML](#) anew, pointing out that it's useful when working with linked servers.

2007-09-23 – Added sections on [global temp tables](#) and [table parameters](#). Reviewed the section on [XML](#).

2005-12-19 – Article revised to cover SQL 2005, and added section on cursor variables.

2005-03-27 – Various minor clarifications on suggestion from Simon Hayes. The bug about INSERT-EXEC and IMPLICIT_TRANSACTIONS is now fixed in SQL 2000 SP4 and SQL 2005.

[Back to my home page.](#)